Blatt 6

Grundlagen der IT-Sicherheit

Luis Staudt

Aufgabe 1: Hashketten

a) Hashkette der Länge 4

Gewählte Hashfunktion: SHA-256. Passphrase $k =$ `LuisStaudt2026`. Die Hashkette wird iterativ gebildet: $P_i = SHA256(P_{i-1})$ mit $P_0 = k$.

```
k   = LuisStaudt2026
P_1 = fc0bdf2471b8dcaeb6100f9b95ce6532
      935dafa4fb4f0ef3abb5c56483aebd71
P_2 = b8359d246c00986878167c1e85bc566c
      b55b25904d1ebcf5a826ca072297536a
P_3 = f245e7df73577ecce17b149c4227329d
      b0baf59459a63fc25b2cc2c9d3fb9deb
P_4 = 19d17121ad7c32e1c3f5fa66f7ad4cdc
      552c9e2fe735be427e7933da9bf7ba0b
```

b) Registrierung und erste Anmeldung

Beim Verfahren nach Lamport wird der höchste Hashwert P_4 initial beim Server registriert. Danach werden die Einmalpasswörter in umgekehrter Reihenfolge (P_3, P_2, P_1) zur Anmeldung verwendet.

Registrierung:

1. Client berechnet die Kette P_1, P_2, P_3, P_4 aus k.
2. Client sendet ausschließlich P_4 an den Server.
3. Server speichert $(\text{user}, P_4, i = 4)$.

Erste Anmeldung:

1. Server fordert das Passwort zum Index $i - 1 = 3$ an.
2. Client sendet P_3.
3. Server prüft $SHA256(P_3) \stackrel{?}{=} P_4$.
4. Ergebnis: stimmt überein $\Rightarrow$ Login erfolgreich.

5. Server ersetzt den gespeicherten Wert durch P_3 und erniedrigt den Index auf $i = 3$.

Wichtig: Aus P_3 lässt sich P_4 berechnen, aber nicht umgekehrt. Ein Angreifer, der P_4 kennt, kann daraus nicht den nächsten benötigten Wert P_3 ableiten.

c) Eve fängt P_{i+1} ab

Eve hat die Verbindung unter Kontrolle und fängt die Antwort P_{i+1} ab, bevor sie Bob erreicht.

- Eve kann sich gegenüber Bob anstelle von Alice anmelden, indem sie P_{i+1} selbst weiterreicht.
- Da Alice die Antwort bereits verschickt hat, ist P_{i+1} das aktuell angeforderte Einmalpasswort.
- Das Passwort bleibt nur für genau *eine* Anmeldung gültig. Sobald Bob P_{i+1} akzeptiert, speichert er diesen Wert und fordert beim nächsten Mal P_i an. P_{i+1} ist damit verbraucht und wertlos.
- Gültigkeit also: bis zur nächsten erfolgreichen Anmeldung mit diesem Passwort; danach nicht mehr nutzbar.

d) Wann fällt der Verlust auf?

Wenn Eve P_{i+1} einmalig nutzt und die Übertragung von Alice zu Bob unterbunden hat, synchronisieren Alice und Bob ihren Index nicht mehr. Der Verlust fällt bei der nächsten regulären Anmeldung von Alice auf:

- Bob hat den Index bereits auf $i-1$ erniedrigt (nach Eves Anmeldung) und erwartet jetzt P_i.
- Alice rechnet davon aus, dass P_{i+1} noch nicht angekommen ist, und versucht diesen Wert erneut oder den nächsten in ihrer Zählung.
- Dadurch kommt es zu einem Index-Mismatch: Alice sendet einen Hash, dessen Prüfung bei Bob fehlschlägt.

Der Angriff wird also spätestens bemerkt, sobald Alice das nächste Mal einen Anmeldeversuch unternimmt.

Aufgabe 2: Rechtemanagement unter Linux

Alle Befehle werden in `/tmp/test` ausgeführt. Die Benutzer alice, bob und eve sowie Testdateien werden wie in der Vorbereitung beschrieben angelegt.

Vorbereitung

```
cd /tmp && mkdir test && cd test
sudo useradd alice
sudo useradd bob
sudo useradd eve
cat /etc/passwd | grep -E "alice|bob|eve"
alice:x:1001:1001::/home/alice:/bin/sh
bob:x:1002:1002::/home/bob:/bin/sh
eve:x:1003:1003::/home/eve:/bin/sh

echo "Geheimer Text von Alice"       > alice.txt
echo "Geheimer Text von Bob"         > bob.txt
echo "Alice und Bob teilen sich das" > alicebob.txt
echo "Für alle sichtbar"             > world.txt
```

a) Interpretation von `ls -l`

```
drwxrwxr-x  2 luis luis  4096 Apr 15 23:23 ./
-rw-rw-r--  1 luis luis    30 Apr 15 23:23 alicebob.txt
-rw-rw-r--  1 luis luis    24 Apr 15 23:23 alice.txt
-rw-rw-r--  1 luis luis    22 Apr 15 23:23 bob.txt
-rw-rw-r--  1 luis luis    19 Apr 15 23:23 world.txt
```

Spaltenbedeutung: Rechtemaske | Anzahl Hardlinks | Besitzer | Gruppe | Größe | Datum | Name.

Die Rechtemaske besteht aus einem Dateityp (`-` = Datei, `d` = Verzeichnis) und drei Gruppen zu je drei Bits für *owner*, *group* und *others* mit jeweils `r` (lesen, 4), `w` (schreiben, 2) und `x` (ausführen, 1).

Alle Testdateien gehören aktuell `luis:luis` mit der Maske `-rw-rw-r-` (664). Das heißt: Besitzer und Gruppe dürfen lesen und schreiben, alle anderen dürfen nur lesen.

b) Besitzverhältnisse mit `chown` ändern

```
sudo chown alice:alice alice.txt
sudo chown alice:alice alicebob.txt
sudo chown bob:bob     bob.txt
sudo chown bob:bob     world.txt
ls -l
-rw-rw-r-- 1 alice alice 30 Apr 15 23:23 alicebob.txt
-rw-rw-r-- 1 alice alice 24 Apr 15 23:23 alice.txt
-rw-rw-r-- 1 bob   bob   22 Apr 15 23:23 bob.txt
-rw-rw-r-- 1 bob   bob   19 Apr 15 23:23 world.txt
```

c) Alice liest `bob.txt`?

```
sudo su alice
$ whoami
alice
$ cat bob.txt
Geheimer Text von Bob
$ cat alice.txt
Geheimer Text von Alice
```

Alice kann `bob.txt` lesen, obwohl die Datei Bob gehört. Grund: Die Rechtemaske `-rw-rw-r-` gibt allen anderen Benutzern (*others*) Leserechte. Alice fällt für `bob.txt` unter *others* und darf deshalb den Inhalt sehen. Erst mit strikteren Rechten (600) wird der Zugriff verweigert.

d) Zugriff nur für den Besitzer (600)

```
sudo chmod 600 alice.txt bob.txt alicebob.txt world.txt
ls -l
-rw------- 1 alice alice 30 Apr 15 23:23 alicebob.txt
-rw------- 1 alice alice 24 Apr 15 23:23 alice.txt
-rw------- 1 bob   bob   22 Apr 15 23:23 bob.txt
-rw------- 1 bob   bob   19 Apr 15 23:23 world.txt
```

Prüfung als Alice, Bob und Eve:

```
sudo su alice
$ cat alice.txt
Geheimer Text von Alice
$ cat bob.txt
cat: bob.txt: Permission denied

sudo su bob
$ cat bob.txt
Geheimer Text von Bob
$ cat alice.txt
cat: alice.txt: Permission denied

sudo su eve
$ cat alice.txt
```

```
cat: alice.txt: Permission denied
$ cat bob.txt
cat: bob.txt: Permission denied
```

Der Zugriff wird jetzt bei allen fremden Benutzern korrekt verweigert.

e) `alicebob.txt` gemeinsam für Alice und Bob

Eine gemeinsame Gruppe `aliceundbob` wird angelegt, Alice und Bob werden Mitglieder. Die Datei erhält diese Gruppe als Gruppenbesitzer und die Maske `660`.

```
sudo groupadd aliceundbob
sudo usermod -a -G aliceundbob alice
sudo usermod -a -G aliceundbob bob
sudo chown alice:aliceundbob alicebob.txt
sudo chmod 660 alicebob.txt
ls -l
-rw-rw---- 1 alice aliceundbob 30 Apr 15 23:23 alicebob.txt
-rw------- 1 alice alice       24 Apr 15 23:23 alice.txt
-rw------- 1 bob   bob         22 Apr 15 23:23 bob.txt
-rw------- 1 bob   bob         19 Apr 15 23:23 world.txt
```

Prüfung (Login-Shell `su -`, damit die neue Gruppenzugehörigkeit aktiv ist):

```
sudo su - alice -c "cat /tmp/test/alicebob.txt; echo 'NEU-Alice' >> /tmp/test/alicebob.txt &&
    echo OK-write"
Alice und Bob teilen sich das
OK-write

sudo su - bob   -c "cat /tmp/test/alicebob.txt; echo 'NEU-Bob'   >> /tmp/test/alicebob.txt &&
    echo OK-write"
Alice und Bob teilen sich das
NEU-Alice
OK-write

sudo su - eve   -c "cat /tmp/test/alicebob.txt"
cat: /tmp/test/alicebob.txt: Permission denied
```

Alice und Bob dürfen lesen und schreiben, Eve nicht.

f) `world.txt` für alle

```
sudo chmod 666 world.txt
ls -l
-rw-rw---- 1 alice aliceundbob 48 Apr 15 23:29 alicebob.txt
-rw------- 1 alice alice       24 Apr 15 23:23 alice.txt
-rw------- 1 bob   bob         22 Apr 15 23:23 bob.txt
-rw-rw-rw- 1 bob   bob         19 Apr 15 23:23 world.txt

sudo su - eve -c "cat /tmp/test/world.txt; echo 'EVE war hier' >> /tmp/test/world.txt && cat /
    tmp/test/world.txt"
Für alle sichtbar
```

```
Für alle sichtbar
EVE war hier
```

Mit 666 dürfen Besitzer, Gruppe und alle anderen lesen und schreiben. Eve kann die Datei erfolgreich ändern.

Aufgabe 3: SAML und Identitäts-Föderation

Gegenstand der Untersuchung ist die Anmeldung am Dienst `bwsyncandshare.kit.edu` im Inkognito-Modus. Die Analyse erfolgt anhand einer HAR-Mitschrift aus den Entwicklertools.

a) Warum kann ich mich am KIT-Dienst mit HFU-Account anmelden?

bwSync&Share wird vom KIT betrieben, ist aber Teil der landesweiten Föderation *bwIDM*. Der Dienst nimmt nur die Rolle des Service Providers (SP) ein und lagert die Authentifizierung an einen Identity Provider (IdP) aus. Beim Login wählt der Nutzer seine Hochschule; der SAML-Standard sorgt dafür, dass der HFU-IdP (`idp2.hs-furtwangen.de`) die Identität bestätigt. Nach erfolgreicher Prüfung übermittelt der IdP eine signierte SAML-Assertion an den SP, der daraufhin eine lokale Sitzung eröffnet.

Die eigentlichen Zugangsdaten verlassen dabei nie die HFU. Das KIT vertraut ausschließlich der Signatur des HFU-IdPs.

b) Host-Kette beim Aufruf von bwSync&Share

Im Inkognito-Fenster werden nacheinander folgende Hosts angesprochen (Reihenfolge aus HAR):

1. `bwsyncandshare.kit.edu`: SP, erzeugt initialen `SAMLRequest` und leitet weiter.
2. `bwidm.scc.kit.edu`: Discovery-/Shibboleth-SP der bwIDM-Föderation, forwarded zum IdP der gewählten Hochschule.
3. `idp2.hs-furtwangen.de`: HFU-IdP, erzeugt eigenen `SAMLRequest`, verlangt Login.
4. `login.microsoftonline.com`: Azure AD der HFU, hier erfolgt die eigentliche Eingabe von Benutzername, Passwort und MFA.
5. Rückweg: Azure AD → HFU-IdP → bwIDM-SP → bwSync&Share mit signierter SAML-Assertion.

Auffällig: Der HFU-IdP delegiert die Anmeldung selbst an Microsoft Azure AD weiter. Shibboleth fungiert hier nur noch als SAML-Frontend, die Identitätsprüfung erfolgt im Microsoft-Tenant der HFU.

Übertragene Daten (Auszug):

Initialer SAML-Request vom SP (bwSync&Share) an bwIDM:

```
<samlp:AuthnRequest
    ID="ONELOGIN_f207b0bd03a182e31c4af2883dc21bc687b5a15b"
    Version="2.0" IssueInstant="2026-04-15T20:32:52Z"
    Destination="https://bwidm.scc.kit.edu/saml/idp/redirect"
    ProtocolBinding="urn:oasis:names:tc:SAML:2.0:bindings:HTTP-POST"
    AssertionConsumerServiceURL="https://bwsyncandshare.kit.edu/apps/user_saml/saml/acs">
  <saml:Issuer>
    https://bwsyncandshare.kit.edu/apps/user_saml/saml/metadata
  </saml:Issuer>
  <samlp:NameIDPolicy
    Format="urn:oasis:names:tc:SAML:1.1:nameid-format:unspecified"
    AllowCreate="true" />
</samlp:AuthnRequest>
```

Nach der Weiterleitung durch bwIDM folgt ein zweiter, getrennter SAML-Request an den HFU-IdP:

```
<saml2p:AuthnRequest
    AssertionConsumerServiceURL="https://bwidm.scc.kit.edu/Shibboleth.sso/SAML2/POST"
    Destination="https://idp2.hs-furtwangen.de/idp/profile/SAML2/Redirect/SSO"
    ID="b8cfd11c8-2f79-4607-88bb-7924feff9643"
    IssueInstant="2026-04-15T20:33:10.344Z"
    ProtocolBinding="urn:oasis:names:tc:SAML:2.0:bindings:HTTP-POST"
    Version="2.0">
  <saml2:Issuer>https://bwidm.scc.kit.edu/sp</saml2:Issuer>
  <saml2p:NameIDPolicy AllowCreate="true"/>
</saml2p:AuthnRequest>
```

Die Antwort des IdP ist eine verschlüsselte SAML-Assertion, die per Hidden-Form und automatischem POST an den `AssertionConsumerService` des SPs gesendet wird.

Wichtigste Cookies im Ablauf:

- `saml_data` (bwSync&Share): merkt sich den Zustand des ausgehenden SAML-Requests.
- `JSESSIONID` (bwIDM, HFU-IdP): Session des jeweiligen Java-Servers während des Login-Flows.
- `__Host-shib_idp_session` (HFU-IdP): die zentrale SSO-Session beim IdP.
- `nc_session_id` (bwSync&Share): lokale Nextcloud-Session nach erfolgreichem Login.

c) Warum kein erneuter Login bei Felix?

Nach dem erfolgreichen Login bei bwSync&Share hat der Browser das Cookie `__Host-shib_idp_sessio` beim HFU-IdP gesetzt. Dieses Cookie bindet die aktive IdP-Sitzung an den Browser.

Ruft der Nutzer anschließend Felix auf, beginnt die gleiche SAML-Kette: Felix sendet einen `SAMLRequest` an denselben IdP. Der IdP erkennt die laufende Sitzung anhand des Cookies und überspringt die Anmeldemaske komplett. Er erzeugt sofort eine neue SAML-Assertion für den neuen SP (Felix). Aus Nutzersicht entsteht so der Eindruck eines Single-Sign-On.

Wichtig: Jeder SP hat seine eigene lokale Sitzung (z. B. `_shibsession` bei Felix, `nc_session_id` bei bwSync&Share). Die gemeinsame Komponente über Dienste hinweg ist ausschließlich die IdP-Session.

d) Vergleich zu Kerberos

Sowohl Kerberos als auch SAML realisieren Single-Sign-On mittels eines vertrauten Dritten. Parallelen:

- Kerberos KDC (Key Distribution Center, Authentication Server + TGS) entspricht dem SAML-IdP.
- Kerberos-Dienst (z. B. Fileserver) entspricht dem SAML-SP.
- Das *Ticket-Granting-Ticket* (TGT), welches nach der initialen Passwortprüfung beim KDC ausgestellt wird, entspricht am ehesten dem Cookie `__Host-shib_idp_session` beim IdP: Beide dienen als Nachweis einer aktiven Anmeldung und werden für weitere Authentifizierungen wiederverwendet.
- Das pro Dienst ausgestellte *Service-Ticket* entspricht der einzelnen SAML-Assertion, die der IdP für jeden einzelnen SP erzeugt.

Hauptunterschied: Kerberos arbeitet nur im lokalen Netz mit symmetrischer Kryptografie, SAML nutzt asymmetrische Signaturen über HTTP und ist deshalb auch Cross-Organisation einsetzbar.